

主要思想

prompt_worker里面接收到一个请求的时候，调用PromptExecutor的execute执行。

```
# main.py的prompt_worker里面
e.execute(item[2], str(uuid.uuid4()), item[3], item[4])

# execution.py的PromptExecutor
# prompt: json字典构成的一个计算图graph;
# prompt_id: 和前端交互的任务id;
# extra_data: 保存到png图片时，另外存的一些额外信息，这些信息可以用来复现工作流的；
# execute_outputs: 需要计算的输出节点
def execute(self, prompt, prompt_id, extra_data={}, execute_outputs=[]):
```

主要思想：

- 前端以json字典(prompt)的方式发送一个计算图给后端，后端在执行的过程中通过prompt_id和前端进行交互。
- 在这个计算图中，从结果节点出发，通过深度优先的方式构建图（ExecutionList），并确定节点的执行顺序。
- 按照执行顺序对所有待执行的节点进行执行：
 - 如果节点未缓存，那么创建节点，并放入到objects缓存中。
 - 如果结果未缓存，那么调用节点的FUNCTION方法执行，并放入到outputs缓存中。
 - 缓存方式：node_id --> 通过keyset获得实际的key --> 通过实际的cache得到最终的缓存

prompt

看个例子。json和工作流的截图

关键类的定义

```
class PromptExecutor:
    def __init__(self):
        self.caches_outputs = {}
        self.caches_objects = {} # key并不直接是node_id，而是CacheKeySet中定义的key
        # 用法：
        # node_id = "1"
        # key_set = CacheKeySet([])
        #
        # output_key_1 = self.caches_outputs_key_set[node_id]
        # output_1 = self.caches_outputs[output_key_1]
        # node_id是不能直接作为key的，这里通过CacheKeySet把node_id转成另一种key

# 它生成上面self.caches_objects对应的key，用来缓存节点对象
# self.caches_objects的key就直接用class_name就可以了，因为comfyui的节点类的__init__函数
# 都是没有参数的，所以每个节点类创建出来的节点对象都相同，所以可以直接用class_name作为key来缓存
# 一个节点对象。
```

```
# execution.py里面的: obj = class_def()
class CacheKeySetID:
    def __init__(self):
        pass

# 它生成的key用来缓存节点对象的执行结果。
# 因为节点执行结果 和 节点输入有关, 如果输入变了, key也要跟着变, 所以这里的key需要递归地把当前
# 节点 以及 所有祖先节点的IS_CHANGE信息加入到key中。当当前节点 以及 所有祖先节点的输入都没变的时
# 候, 才可以命中缓存。
# IS_CHANGE: 根据输入的内容计算的一个值, 可以是任意可以hash的内容(因为要放到key里面), 不一定
# 是BOOL
class CacheKeySetInputSignature:
    def __init__(self):
        pass

class ExecutionList:
    """
    ExecutionList 实现了一种图拓扑的解法。当一个节点被标记为可执行的时候, 如果有更进一步的关
    系被添加时, 这个节点仍然可以变为graph的其他状态(非准备状态)。
    """
    def __init__(self, dynprompt, output_cache):
        self.dynprompt = dynprompt
        self.pendingNodes = {} # key: node_id, value: 是否等待中。其实用个list或者set
        # 来替代会好一点。只要node_id在这个dict中, 那么就是在等待, 否则就不是在等待。
        self.blockCount = {} # key: node_id, value: 这个节点的被多少个先前的节点阻塞
        # 住, 如果是0, 那么就可以执行。
        self.blocking = {} # key: node_id, value: dict。value记录这个节点的第i个输出
        # 阻塞了哪些后续节点。比如, self.blocking[0][1][2] = True, 表示第0个节点的第2个输出, 阻塞了第
        # 1个节点。
        self.output_cache = output_cache # 如果这个节点的执行结果已经在缓存中了, 那么就
        # 不再执行了
        self.staged_node_id = None
```

执行伪代码

```
# PromptExecutor的方法
# prompt: 前端发送的json字典计算图
# execute_outputs: 需要计算的输出节点
def execute(self, prompt: dict, execute_outputs=[]):

    objects_key_set = CacheKeySetID(prompt) # 计算当前任务每个node_id对应的key
    outputs_key_set = CacheKeySetInputSignature(prompt)

    self.objects_cache.clean(objects_key_set) # 清除缓存
    self.outputs_cache.clean(outputs_key_set)

    execute_list = ExecutionList(prompt, self.outputs_cache, outputs_key_set)
    execute_list.add(execute_outputs)

    while not execute_list.empty():
        node_id_to_execute = execute_list.pop()

        execute(node_id_to_execute, self.outputs_cache, outputs_key_set,
self.objects_cache, objects_key_set) # 执行节点。通过node的FUNC来执行
```

